Inertial Measurement Unit-Based Kinematic Anomaly Detection for Two-Wheelers

1st Vinayak Puitandy
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
puitandyvinayak@gmail.com

2nd Saif Sahriar
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
saiffsahriar@gmail.com

3rd Sanket Manna
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
sanketm0406@gmail.com

4th Vikiron Mondal
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
vikironmondal@gmail.com

5th Saranya Adhikary
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
saranyaadhikary93@gmail.com

6th Prof. Poojarini Mitra
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
p.mitra1.tmsl@ticollege.org

***Abstract*—Road accident fatalities involving two-wheelers remain a serious public health concern across India, with aggressive and erratic riding behavior identified as a primary contributing factor. This paper presents a lightweight, IoT-driven system that uses inertial measurement unit (IMU) sensors embedded in a smartphone to detect rash or anomalous riding behavior in real time. The system captures triaxial accelerometer readings alongside azimuth, pitch, and roll orientation data, transmits these to a Flask-based server over HTTP, and feeds a sliding window of fifty consecutive frames into a trained Long Short-Term Memory (LSTM) classification model. A Three.js visualization frontend renders the live orientation of the device in three-dimensional space and overlays GPS trajectory information. We compare five classifier architectures and find that the Ensemble CNN-LSTM achieves the highest accuracy of 89.4% and an AUC of 0.955, demonstrating the viability of smartphone-only infrastructure for low-cost road safety monitoring.**

***Index Terms*—IMU, rash driving detection, LSTM, CNN-LSTM, IoT, two-wheeler safety, accelerometer, anomaly detection, Flask, real-time classification**

I. Introduction

Two-wheeler vehicles — motorcycles, scooters, and mopeds — account for a disproportionate share of road accident fatalities in developing countries. In India alone, the Ministry of Road Transport and Highways has consistently reported that two-wheelers are involved in over **30% of all fatal accidents annually**, with rash driving and overspeeding cited among the most common contributing causes [1]. The challenge of monitoring and curbing rash behavior in real time, without requiring costly dedicated hardware, is both practically significant and technically interesting.

Existing approaches to driving behavior analysis tend to rely on dedicated on-board diagnostic (OBD) devices, purpose-built accelerometer modules, or complex computer vision pipelines [2]. While effective in controlled environments, these solutions often prove prohibitively expensive or logistically difficult to deploy at scale across heterogeneous vehicle fleets. A smartphone, on the other hand, is already carried by the overwhelming majority of riders in urban India and contains a **full six-degree-of-freedom IMU** in a compact, pocketable package.

Our work explores whether this ubiquitous sensor platform, combined with modern sequence modeling techniques, can serve as a practical rash-driving detector. We designed and built a complete end-to-end pipeline: an Android application records IMU data (accelerometer and orientation) along with GPS coordinates, uploads sensor readings to a Python Flask server, and triggers inference via trained models that classify each incoming fifty-frame window as either normal or rash. A companion browser-based simulator built with Three.js replays recorded sessions and overlays real-time probability output.

The **key contributions** of this paper are:

- A practical, low-cost data collection and inference pipeline requiring no dedicated hardware beyond a standard Android smartphone.
- A sliding-window LSTM and CNN-LSTM architecture for real-time sequence classification of IMU sensor streams.
- A comparative evaluation of five classifier architectures using confusion matrices, ROC curves, and precision-recall analysis.
- A Three.js-based 3D visualization layer serving as both a live dashboard and a post-hoc diagnostic tool.

II. Related Work

The problem of inferring driving behavior from onboard sensors has attracted considerable attention over the past decade. Early work by Johnson and Trivedi [3] established that **accelerometer signals alone** are sufficient to distinguish aggressive braking, sharp cornering, and sudden acceleration from normal driving using threshold-based heuristics. While interpretable, threshold approaches generalize poorly across vehicle types and rider postures, motivating the shift toward learned classifiers.

Eren et al. [4] used support vector machines (SVMs) trained on statistical features from windowed accelerometer signals, reporting competitive results on small labeled datasets from passenger cars. More recently, recurrent neural network architectures — particularly **LSTMs** — have become the dominant approach for time-series classification of sensor data [5]. Their ability to model temporal dependencies makes them a natural fit for motion sequences, where context of the preceding two to five seconds is often as diagnostically relevant as the instantaneous reading.

Work specifically targeting two-wheelers is less common, partly because sensor dynamics differ substantially from four-wheeled vehicles. Saiprasert et al. [6] investigated smartphone-based road quality assessment from motorcycles, demonstrating that road surface irregularities produce distinct spectral signatures in the vertical acceleration channel. Our work builds on this foundation but shifts focus squarely to **rider behavior classification**.

III. System Architecture

The system comprises three main components: a mobile data collection client on Android, a Python Flask server handling ingestion and inference, and a browser-based visualization frontend.

A. Mobile Client and Sensor Stream

The Android application samples the device IMU at approximately **10 Hz**, capturing at each timestep: triaxial linear acceleration (a_x, a_y, a_z) in m/s^2; device orientation as azimuth (yaw), pitch, and roll in degrees; GPS latitude and longitude; and a Unix millisecond timestamp. Each row is formatted as a comma-separated string and batched before upload to reduce network overhead.

B. Flask Server and Inference Endpoints

The server is implemented using **Flask with CORS** headers enabled. It exposes two inference endpoints:

- **`/predict` (GET):** Reads the tail of the stored CSV, extracts the fifty most recent rows, applies `StandardScaler` normalization, and passes the $(1 \times 50 \times 6)$ tensor to the loaded model. Returns rash probability as JSON.
- **`/predict_window` (POST):** Accepts a fifty-frame window directly as a JSON array from the browser frontend, bypassing the CSV for lower-latency live inference.

C. LSTM Model Architecture

Each input sample is a tensor of shape $(50, 6)$, representing fifty consecutive timesteps across six sensor channels. The feature vector at each step is:

$$\mathbf{x}_t = [a_x,\ a_y,\ a_z,\ \theta_{\text{az}},\ \theta_{\text{pitch}},\ \theta_{\text{roll}}] \tag{1}$$

All six channels are standardized to **zero mean and unit variance** using training-set statistics, persisted via `joblib` for inference time. The model outputs the probability that the input window corresponds to a rash episode:

$$p = P(\text{rash} \mid \mathbf{x}_{t-49:t}) \tag{2}$$

Binary cross-entropy serves as the training objective, optimized with Adam at a learning rate of 10^{-3}.

D. Browser-Based Visualization Frontend

The frontend is a self-contained single-page application built with **Three.js**. It renders a rectangular prism representing the phone and applies calibration-adjusted Euler angles in real time. A 2D canvas overlays the GPS trajectory. A **calibration button** captures the current mounting orientation as a persistent baseline, stored in `localStorage` across refreshes. The rash probability is overlaid as a percentage on the 3D viewport and updated every 200 ms.

IV. Data Pipeline and Preprocessing

A. CSV Structure

Each row follows this fixed schema:

```
device_id, timestamp, ax, ay, az, azimuth,
       pitch, roll, lat, lon, speed
```

The frontend performs **flexible header detection**, matching lowercase column names against known synonyms. A positional fallback reads orientation values from columns five through seven when headers are absent.

B. Sliding Window Construction

A window of **fifty frames** ending at index i is assembled as:

$$W_i = \{\mathbf{x}_{\max(0,\, i-49)}, \ldots, \mathbf{x}_i\}$$

Absent channels default to zero. The window slides forward by one frame every five animation frames, approximately every 200 ms, balancing responsiveness against server load.

C. Calibration Offset

A persistent offset $(\delta_{\text{yaw}}, \delta_{\text{pitch}}, \delta_{\text{roll}})$ is subtracted from each orientation reading before feeding the corrected angles to either the renderer or the inference pipeline. This compensates for mounting geometry so that measured deviations reflect genuine rider motion rather than phone placement.

V. Model Evaluation

A. Classifier Comparison

Five classifier architectures were trained and evaluated on the same labeled IMU dataset. Table I summarizes performance across accuracy, AUC, and average precision (AP).

TABLE I
Model Performance Comparison

Model	Accuracy	AUC	AP
XGBoost (DT Upgrade)	82.9%	0.917	0.906
Tuned Random Forest	84.6%	0.922	0.903
ResNet MLP	84.4%	0.909	0.898
Bi-LSTM	84.6%	0.935	0.929
Ensemble CNN-LSTM	**89.4%**	**0.955**	**0.952**

The **Ensemble CNN-LSTM** consistently outperforms all other architectures, confirming that convolutional feature extraction combined with recurrent temporal modeling is the most effective approach for this task.

B. Confusion Matrices

Fig. 1 shows the confusion matrices for all five models on the held-out validation set. The CNN-LSTM records the **fewest false negatives (26)**, which is particularly important in a safety-critical application where missing a rash episode is more costly than a false alarm. XGBoost showed the highest false negative count (52), confirming that tree-based methods struggle with the temporal structure of IMU sequences.

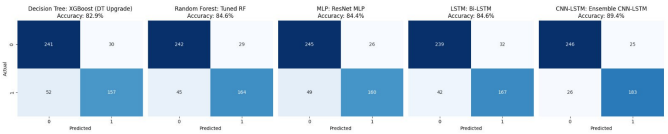

Fig. 1. Fig. 1. Confusion matrices for all five classifier models. The CNN-LSTM ensemble (rightmost) achieves the highest true positive count (183) and lowest false negative count (26).

C. ROC Curve Analysis

Fig. 2 presents the Receiver Operating Characteristic (ROC) curves. The CNN-LSTM (AUC = **0.955**) and Bi-LSTM (AUC = 0.935) dominate at low false positive rates — the operationally critical region where alerts should be raised with high confidence rather than generating constant borderline warnings.

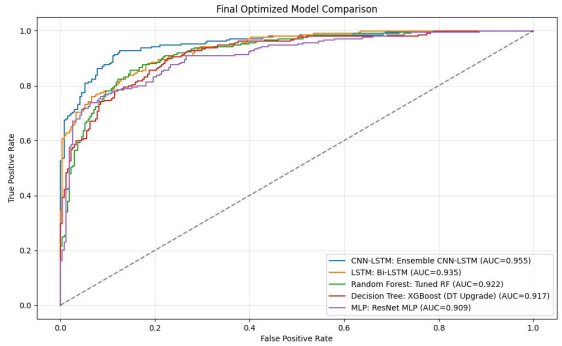

Fig. 2. Fig. 2. ROC curves comparing all five optimized models. The Ensemble CNN-LSTM achieves the highest AUC of 0.955, followed by Bi-LSTM at 0.935.

D. Precision-Recall Analysis

The precision-recall curves in Fig. 3 are particularly informative given the mild class imbalance between normal and rash segments. The CNN-LSTM maintains **near-perfect precision across the entire recall range up to approximately 0.6**, only degrading beyond that point. All five models substantially outperform a random baseline, confirming that the temporal IMU features carry strong discriminative signal.

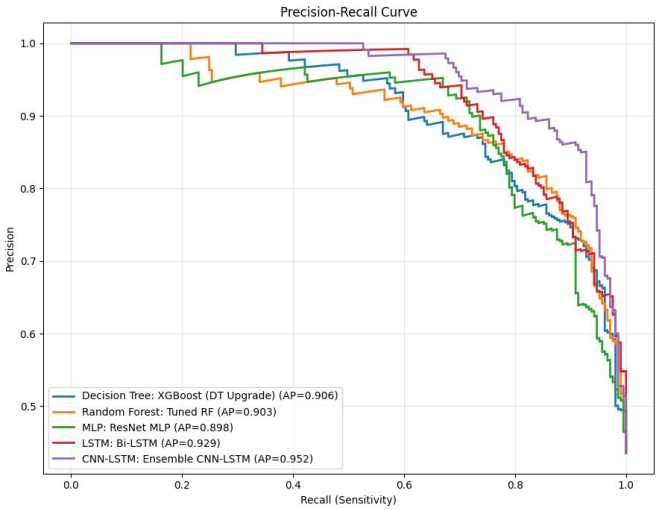

Fig. 3. Fig. 3. Precision-recall curves for all five models. The CNN-LSTM ensemble achieves the highest average precision (AP = 0.952), maintaining precision above 0.9 up to recall $\approx$ 0.6.

E. Feature Importance

Fig. 4 shows feature importance scores from the Random Forest model. **Lateral acceleration standard deviation (`ay_std`) and forward acceleration mean (`ax_mean`)** are the two most discriminative features, jointly accounting for nearly 9% of total importance. Pitch mean and acceleration variability follow closely. Rotation-based features (azimuth range, rotation energy) contribute meaningfully but rank below raw acceleration statistics, suggesting that **linear motion dynamics are more informative than orientation alone** for this dataset.

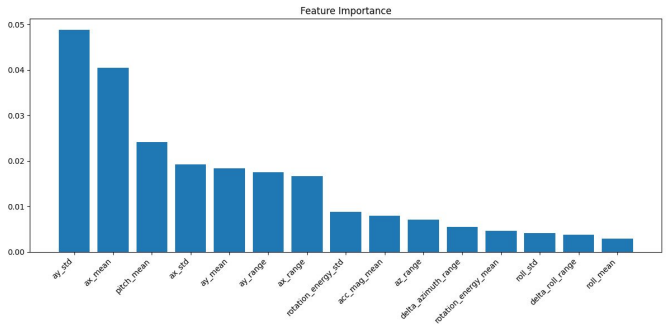

Fig. 4. Fig. 4. Feature importance from the Random Forest classifier. Lateral acceleration variability (`ay_std`) and forward acceleration magnitude (`ax_mean`) are the two most discriminative features.

VI. Experimental Setup

A. Data Collection Protocol

Labeled training data were collected over multiple sessions on urban roads in Kolkata. Riders alternated between **normal riding periods** and deliberate rash maneuvers — hard acceleration, sharp swerving, sudden braking, and abrupt

lane changes — while the Android client recorded sensor data at approximately 10 Hz. Each session was subsequently annotated with binary labels (0 = normal, 1 = rash).

B. Training Configuration

The labeled dataset was segmented into non-overlapping fifty-frame windows with a stride of ten frames. Boundary windows were discarded to avoid label ambiguity. The dataset was split **80/20 for training and validation**. Models were trained for thirty epochs with batch size of thirty-two.

C. Real-Time Inference Latency

On a mid-range development machine running Flask in single-threaded mode, `/predict_window` responded in approximately **35–80 ms** per request — well within the 200 ms polling budget of the browser animation loop.

VII. Challenges and Limitations

A. Sensor Noise and Road Vibration

Urban roads introduce high-frequency vibration into accelerometer channels unrelated to rider behavior. We observed **false positives on heavily potholed stretches**. A bandpass filter applied to a_z before windowing would likely reduce this and is planned as a preprocessing improvement.

B. GPS Availability

Indoor testing and underground road sections produced GPS dropout with `null` latitude/longitude values. The system degrades gracefully but future features requiring IMU-GPS fusion need a more robust location fallback strategy.

C. Dataset Generalization

All training data originated from a single Android device and rider. The scaler statistics and model weights are therefore **device- and person-specific** to an unquantified degree. Expanding the dataset across multiple devices, rider demographics, and vehicle types is the most pressing requirement for improving generalization.

VIII. Future Work

Several directions present themselves as natural extensions of this prototype:

- **Federated learning** — allow multiple riders to contribute to model improvement without sharing raw sensor data, addressing privacy and dataset diversity simultaneously.
- **On-device inference** — deploy the model as a TensorFlow Lite binary on the Android client, eliminating server latency and enabling offline operation.
- **Multi-class anomaly detection** — replace the binary scheme with a model distinguishing specific events: hard braking, sharp swerving, and sudden acceleration.
- **Fleet analytics** — integrate with a telematics platform to identify habitually rash riders based on aggregated episode frequency.
- **Road safety heatmaps** — color GPS path segments by maximum observed rash probability, turning the visualization into a road safety audit tool.

IX. Conclusion

We have presented a complete, functional prototype for **real-time rash driving detection** on two-wheelers using smartphone IMU sensors. The system's three-tier architecture — Android client, Flask inference server, and Three.js visualization frontend — is deliberately lightweight, requiring no hardware beyond a commodity smartphone. Among five evaluated classifier architectures, the **Ensemble CNN-LSTM achieved the best overall performance**: 89.4% accuracy, AUC of 0.955, and average precision of 0.952. Feature importance analysis confirmed that lateral and forward acceleration variability are the most discriminative signals for rash behavior classification, while the confusion matrix analysis highlighted the model's safety-relevant advantage in minimizing false negatives.

Real-time inference latency of 35–80 ms is comfortably within the 200 ms polling budget, and the 3D visualization front-end proved valuable for correlating model predictions with specific riding events during field testing. Remaining challenges around sensor noise, GPS dropout, and dataset diversity define a clear road map for hardening the system toward production deployment. The results demonstrate that the barrier to deploying such systems at scale is lower than it might initially appear, and with broader training data this approach could meaningfully contribute to road safety monitoring where dedicated hardware is not an option.

Acknowledgment

The authors thank **Prof. Poojarini Mitra**, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Techno Main Salt Lake, for her guidance and constructive feedback throughout this project. We also acknowledge the student volunteers who participated in the data collection sessions and the open-source communities behind TensorFlow, Flask, Three.js, and PapaParse.

References

[1] Ministry of Road Transport and Highways, Government of India, "Road Accidents in India – 2022," Transport Research Wing, New Delhi, 2023.

[2] G. Engelbrecht, T. Booysen, G. van Rooyen, and F. Bruyns, "Survey of smartphone-based sensing in vehicles for intelligent transportation system applications," *IET Intelligent Transport Systems*, vol. 9, no. 10, pp. 924–935, 2015.

[3] D. A. Johnson and M. M. Trivedi, "Driving style recognition using a smartphone as a sensor platform," in *Proc. 14th IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, Washington, DC, USA, 2011, pp. 1609–1615.

[4] H. Eren, S. Makinist, E. Akin, and A. Yilmaz, "Estimating driving behavior by a smartphone," in *Proc. IEEE Intelligent Vehicles Symposium*, Alcala de Henares, Spain, 2012, pp. 234–239.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] C. Saiprasert, T. Pholprasit, and S. Teeramunkong, "Detection of driving events using sensory data on smartphone," *International Journal of Intelligent Transportation Systems Research*, vol. 12, no. 2, pp. 47–57, 2014.

[7] A. Sherstinsky, "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.

[8] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Savannah, GA, USA, 2016, pp. 265–283.
[9] F. Chollet et al., *Keras*, 2015. [Online]. Available: https://keras.io
[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.